CODE HELP NOTES

PyTorch Modern ANN — Explained Top to Bottom

*This is a line-by-line guide to **pytorch_modern_ann.py** — a neural network that predicts diabetes (Yes/No) from 8 health features. Read it top to bottom; every main function is explained in plain words. The big flow is: **load data → wrap it → build model → train with early stopping → evaluate → save & reuse**.*

1. Imports — the tools

Load packages

```
import torch
from torch import nn                        # layers, losses
from torch.utils.data import Dataset, DataLoader
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.model_selection import train_test_split
```

`torch / nn`	**What:** the PyTorch engine and its neural-network toolbox. **Why:** gives tensors, layers (`nn.Linear`), losses, and auto-gradients.
`Dataset, DataLoader`	**What:** helpers that wrap your data and feed it in **batches**. **Why:** batching = the mini-batch training from your notes (Part 6).
`pandas / sklearn`	**What:** pandas reads the CSV; scikit-learn splits, scales, and scores. **Why:** same preprocessing & evaluation tools you already know.

2. Load, split & scale the data

Prepare the data

```
data = pd.read_csv('diabetes.csv')
X = data.iloc[:, 0:8]        # 8 feature columns
Y = data.iloc[:, 8]          # label column (0 = No, 1 = Yes)

X_train, X_test, Y_train, Y_test = train_test_split(
        X, Y, test_size=0.2, random_state=0)

sc = StandardScaler()
X_train = sc.fit_transform(X_train)   # learn avg+spread on TRAIN, then scale
X_test  = sc.transform(X_test)        # reuse the SAME scaler (no leakage)
```

`pd.read_csv / .iloc`	**What:** loads the table, then splits columns into inputs X (cols 0–7) and label Y (col 8). **How:** `iloc[rows, cols]` selects by position.
`train_test_split`	**What:** keeps 80% to train and hides 20% to test. **Why:** an honest exam on unseen data. `random_state=0` makes it repeatable.
`StandardScaler fit_transform / transform`	**What:** rescales features to a fair range. **Golden rule:** `fit_transform` learns from TRAIN only; `transform` reuses it on TEST (no data leakage).

3. Wrap the data: Dataset + DataLoader

PyTorch feeds data through a **Dataset** (knows one sample) and a **DataLoader** (serves shuffled batches). This is how batching happens automatically.

Custom Dataset and loaders

```
class DiabetesDataset(Dataset):
    def __init__(self, features, labels):
        self.X = torch.tensor(features, dtype=torch.float32)
        self.Y = torch.tensor(labels.values, dtype=torch.long)
    def __len__(self):
        return len(self.X)               # how many samples
    def __getitem__(self, idx):
        return self.X[idx], self.Y[idx]  # one (features, label) pair

train_dataset = DiabetesDataset(X_train, Y_train)
test_dataset  = DiabetesDataset(X_test,  Y_test)

train_dataloader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_dataloader  = DataLoader(test_dataset,  batch_size=32, shuffle=False)
```

`__init__`	**What:** converts arrays into tensors once. **Why:** the model only accepts tensors; features are `float32`, labels are `long` (integers 0/1 needed by CrossEntropyLoss).
`__len__ / __getitem__`	**What:** the two methods every Dataset must have — total count, and how to fetch sample *idx*. **How:** the DataLoader calls these behind the scenes.
`DataLoader (batch_size, shuffle)`	**What:** serves data in groups of 32. **Why:** mini-batch training (Part 6). **Shuffle:** ON for train (mix order each epoch), OFF for test (stable scoring).

4. Build the model

The ANN (nn.Sequential)

```
class ANN_model(nn.Module):
    def __init__(self, inputFeatures=8, hidden1=64, hidden2=32, outputFeatures=2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(inputFeatures, hidden1),   # 8  -> 64   (Dense)
            nn.Dropout(),                        # drop 50% (fights overfitting)
            nn.ReLU(),                           # activation
            nn.Linear(hidden1, hidden2),         # 64 -> 32
            nn.Dropout(),
            nn.ReLU(),
            nn.Linear(hidden2, outputFeatures)   # 32 -> 2  (raw class scores)
        )
    def forward(self, x):
        return self.net(x)                       # forward propagation

torch.manual_seed(20)                            # reproducible weights
model = ANN_model()
loss_fn   = nn.CrossEntropyLoss()                # loss for 2+ classes
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

`nn.Sequential / nn.Linear`	**What:** stacks layers in order; `nn.Linear(a,b)` is a Dense layer (a→b neurons). **Flow:** 8 → 64 → 32 → 2.
`nn.Dropout() / nn.ReLU()`	**What:** Dropout randomly switches off neurons (default 50%) to reduce overfitting (Part 8); ReLU is the activation (Part 4).
`outputFeatures=2`	**Note:** this network outputs **2 scores** (one per class), not 1 sigmoid. That is why the loss is `CrossEntropyLoss`, not BCELoss.
`CrossEntropyLoss / Adam / manual_seed`	**What:** loss for multi-class (Part 5); Adam updates weights (Part 6); `manual_seed(20)` fixes randomness so results repeat.

5. Train & test functions

One epoch of training, and evaluation

```
def train(dataloader, model, loss_fn, optimizer):
    model.train()                        # training mode (dropout ON)
    for x, y in dataloader:              # loop over batches
        pred = model(x)                  # 1. forward
        loss = loss_fn(pred, y)          # 2. error
        optimizer.zero_grad()            # 3. clear old gradients
        loss.backward()                  # 4. backprop
        optimizer.step()                 # 5. update weights

def test(dataloader, model, loss_fn):
    model.eval()                         # eval mode (dropout OFF)
    all_loss, accuracy = 0, 0
    for x, y in dataloader:
        pred = model(x)
        all_loss += loss_fn(pred, y).item()
        pred = torch.argmax(pred, dim=1)         # pick the higher-score class
        accuracy += (pred == y).sum().item()     # count correct in batch
    return all_loss/len(dataloader), accuracy/len(dataloader.dataset)
```

`train() – the 5 steps`	**What:** the core loop from Part 11: forward → loss → `zero_grad` → `backward` → `step`, run once per batch.
`torch.argmax(pred, dim=1)`	**What:** the model gives 2 scores per sample; argmax picks the index of the bigger one (0 or 1) = the predicted class.
`(pred==y).sum() .item()`	**What:** compares predictions with truth (True/False), sums the Trues = number correct. `.item()` turns a 1-value tensor into a plain number.

`len(dataloader)` vs `len(...dataset)`	**Why different:** loss is averaged over **batches** (count of batches), accuracy over **all samples** (count of rows).

6. Training loop with early stopping

Repeat epochs, stop when it stops improving

```
epochs, patience = 1000, 10
best_loss, counter = float('inf'), 0

for epoch in range(epochs):
    train(train_dataloader, model, loss_fn, optimizer)
    var_loss, acc = test(test_dataloader, model, loss_fn)

    if var_loss < best_loss:      # improved -> remember, reset counter
        best_loss, counter = var_loss, 0
    else:                         # no improvement -> count up
        counter += 1
    if counter >= patience and acc > 0.81:
        print("Early stopping")
        torch.save(model.state_dict(), 'model.pth')   # save weights
        break
```

`patience / counter`	**What:** early stopping. If the test loss does not improve for **10** epochs in a row (and accuracy is already > 81%), training stops — saving time and avoiding overfitting.
`torch.save( state_dict())`	**What:** saves the model's learned weights to `model.pth`. **Why:** so you can reload and reuse it later without retraining.

7. Final report & predicting a new patient

Evaluate, then load model and predict one sample

```
print(classification_report(Y_test, all_preds))   # precision / recall / F1
print(confusion_matrix(Y_test, all_preds))        # right vs wrong grid

loaded_model = ANN_model()
loaded_model.load_state_dict(torch.load('model.pth'))   # restore weights
loaded_model.eval()

new_sample = [3., 128., 52., 65., 0., 39.6, 0.475, 70.]
t_sample = torch.tensor(sc.transform([new_sample]), dtype=torch.float32)
with torch.no_grad():                             # no gradients for prediction
    pred = loaded_model(t_sample).argmax(1).item() # 0 = No, 1 = Yes
```

`classification_report` `confusion_matrix`	**What:** detailed scoring — precision/recall/F1 per class, plus the grid of correct vs wrong predictions (same sklearn tools from Part 10).
`load_state_dict` / `torch.load`	**What:** rebuilds the model shell, then pours the saved weights back in. **Why:** reuse the trained model anytime.
`sc.transform` + `torch.no_grad()`	**Key:** the new patient must be scaled by the *same* scaler, then wrapped in `no_grad()` for a fast, gradient-free prediction.

The whole code in one line

Read the diabetes CSV → split & scale it → wrap it in a Dataset/DataLoader for batching → build a 2-output ANN with Dropout+ReLU → train with Adam & CrossEntropyLoss using the 5-step loop → stop early when it stops improving → score it → save and reuse the model to predict a new patient.